writeup

Programming Assignment 1 Writeup

Moses Dilts

7-3-2026

Program Overview:

This program acts as a simple window manager that allows the user to open, close and switch between a list of windows. Along with this required functionality, the program has bonus functionality of a list command and an ‘all’ argument for the close command. The program uses a stack to store and organize the currently open windows and attempts to prevent erroneous inputs from causing crashes. The focused window is always the window at the top of the stack.

Commands:

- open <window_num> - opens a new window as window number <window_num>
- close <window_num> - finds and closes the given window if it exists
- close <“all”> - closes all windows and quits the program
- switch <window_num> - switches the currently open window to <window_num> if it exists
- ls - lists the windows currently open as well as the window in focus

Error Prevention:

Although it is not required according to the spec, the program will try its hardest to prevent the user from causing a crash with unexpected or invalid input. The first level of protection comes from reading the commands themselves. If a command is not recognized, the program will ignore it (or print an error message if debug mode is on). If an entered command is valid, the program will also check that the arguments are correct. If an argument is not given where it is required, the program will ignore the command. The last line of defense is implemented in the stack functions themselves.

If a user tries to switch to or close a nonexistent window, the program will do nothing (or print “find failed” if debug mode is enabled). You can enable debug mode in defs.h by switching DEBUG from 0 to 1.

Structure:

As mentioned in the overview, the program implements the window tracking functionality with a stack. This structure was chosen because it lends itself well to keeping track of an ordered list (especially one where you regularly need to add things to the front/top and almost never to the end/bottom). When the user opens a new window, a node is pushed to the top of the stack. When a window is closed, the program searches the stack for a node with the given window number. If such a node can be found, it is freed from the stack. If not, the command is ignored. When the user wants to switch the currently focused window, the switch command follows a similar procedure to the close command: If there is a node matching the given argument, the program removes it from the stack and pushes it to the top, making it the focused window. The command is ignored if the target window cannot be found.

Files:

Because this writeup is probably already too long, I’ll keep it brief. The program is separated into two main files: pa1.c and stack.c. These files also have their own headers for imports and function definitions: defs.h and stack.h. I decided to keep things segmented in this way because of the possibility of wanting to re-use the stack code. All parts of the stack should be portable to another project, only needing stack.c and stack.h to be imported.

Mo’s Thoughts / Conclusion:

I enjoyed working on this program overall. Challenging myself to implement a more sophisticated command system than the typical fscanf was my favorite part. If I was to rewrite this program, I would probably break out the command input and validation into its own file to keep things neat, as the huge if-else brick is a little hard to read and add to.